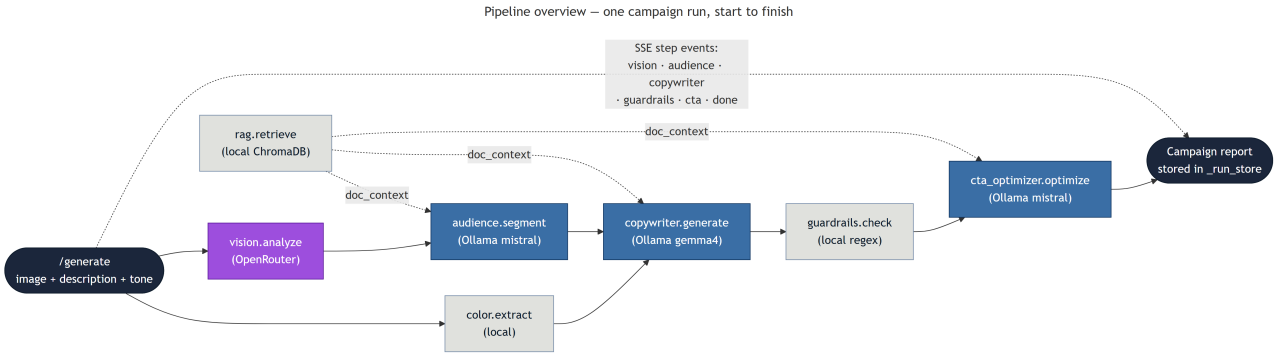
MARKETING AGENT — THE PIPELINE

How one campaign runs, from upload to report

WHAT THIS IS

The pipeline is how the system turns one product image into a finished campaign. It is a fixed sequence of specialist agents: each one has a narrow job, receives only the context it needs, and hands its result to the next. The same agents run in the same order on every campaign.

The reason for the sequence — rather than one big prompt — is the finding in the GAP analysis: an agency splits work across five roles because each phase needs different skills and different context. The pipeline copies that split. DL-02 proves it makes the output better, not just tidier.



See the picture: diagrams/pipeline-overview.mmd

WHY SPECIALISTS, NOT ONE GENERALIST

A single-prompt "monolith" was built first, on purpose, so the comparison would be real. Asked to analyse a product image and write two ad variants in one call, it produced "Check out this amazing product!" for a "general public" audience. The problem is circular: when one call does product analysis, audience inference, and copy at once, it defines the audience in terms of the copy it is already imagining.

Splitting the work breaks that loop. The audience agent's only job is to turn product signals into a persona; the copywriter then writes for a persona it was handed, not one it invented. Same model, same image — the output went from "general public" to "Eco-Conscious Millennial Athlete, 25–34". The gain came from information architecture, not a bigger model.

THE STEPS, IN PLAIN WORDS

1. COLOR — read the palette (local). color.extract pulls a dominant colour and a five-swatch palette from the image with Pillow and ColorThief. No network, instant.

2. VISION — read the product (remote). vision.analyze sends the image to a vision model on OpenRouter and gets back product_type, product_tags, target_signals, mood, and hashtags. It tries a primary model and falls back to a second if the first fails, and records which one answered in model_used.

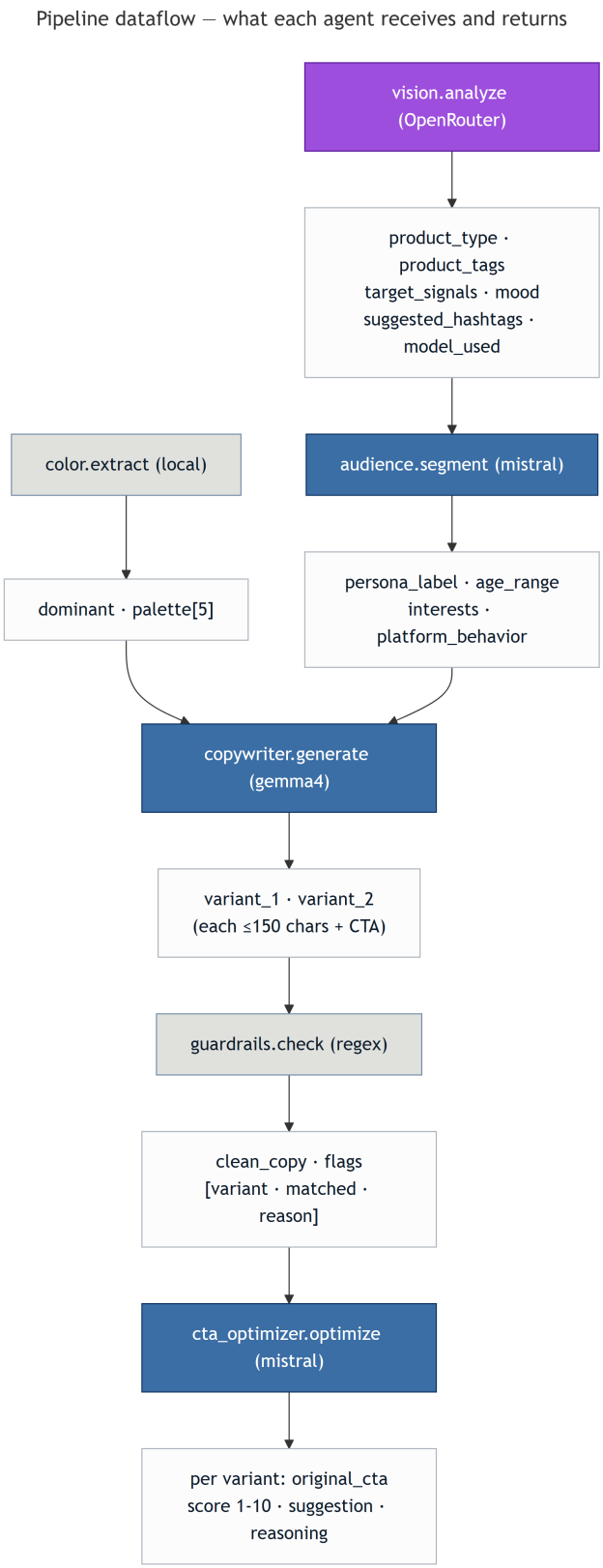
3. RETRIEVE — add document context (local). rag.retrieve searches the ChromaDB knowledge base for chunks relevant to this product and passes them to the text agents, so a campaign can reflect brand guidelines that are not visible in the image.

4. AUDIENCE — build the persona (Ollama mistral). audience.segment turns the description and vision signals into one persona: persona_label, age_range, interests, platform_behavior.

5. COPYWRITER — write two variants (Ollama gemma4). copywriter.generate writes variant_1 (an emotional hook) and variant_2 (a product benefit), each 150 characters or fewer with a clear CTA, written for the persona and the chosen tone.

6. GUARDRAILS — check the claims (local regex). guardrails.check scans both variants for seven categories of risky claim and returns clean_copy plus a list of flags. It does not rewrite the copy; it reports.

7. CTA_OPTIMIZER — score the calls to action (Ollama mistral). cta_optimizer.optimize reads each variant's call to action and returns a score out of 10, a suggested stronger CTA, and the reasoning.



See the picture: diagrams/pipeline-dataflow.mmd

THE MODEL SPLIT

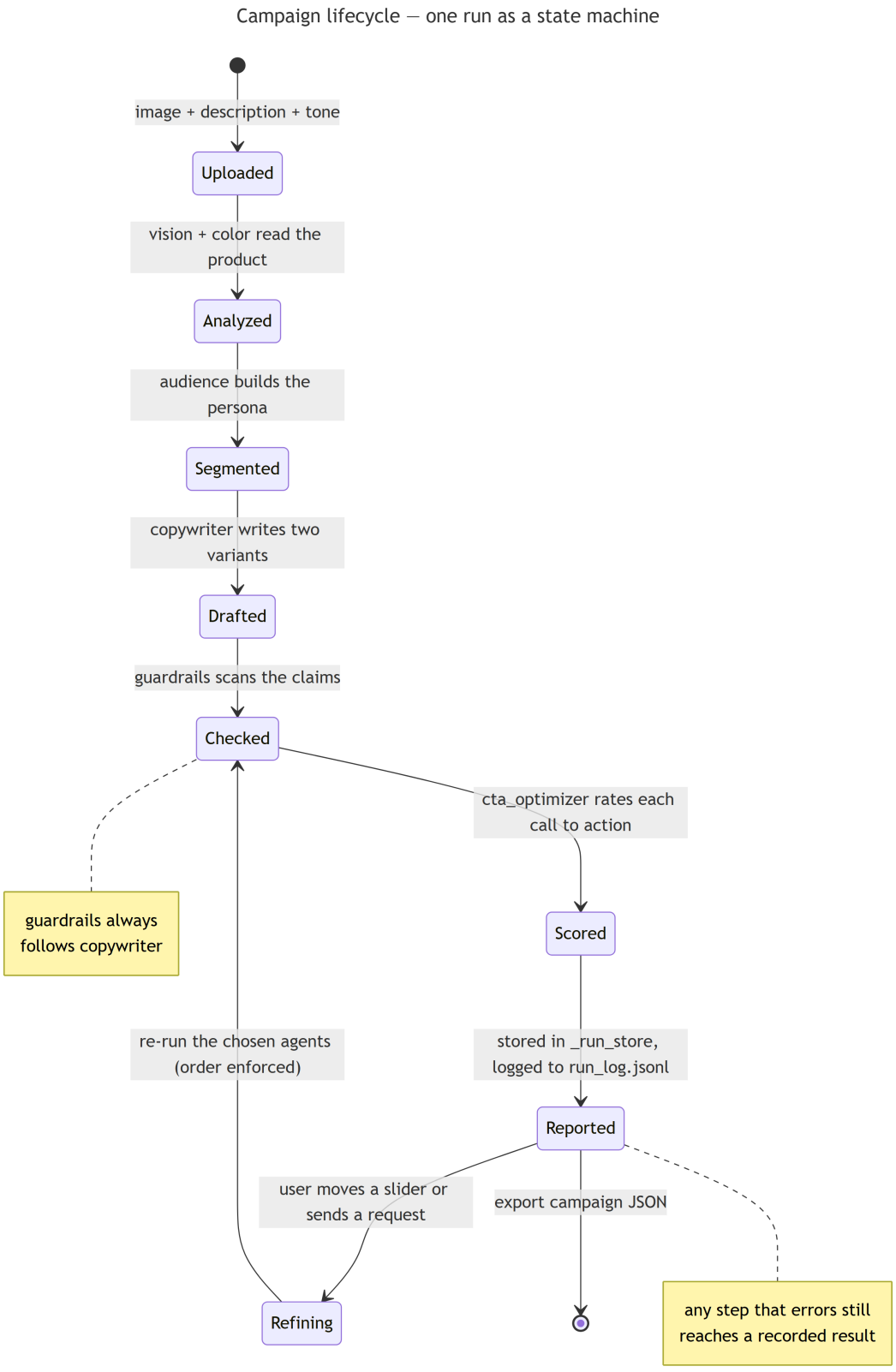
Three model roles, chosen by task, not one model everywhere:

- Vision runs remotely on OpenRouter's free tier, because a local vision model on CPU was far too slow.
- Creative copy uses Ollama gemma4:e2b, which writes more varied, stronger prose and stays fast.
- Structured JSON work (audience, CTA, the refinement router) uses Ollama mistral:7b, which is more reliable at returning clean JSON.
- Embeddings for RAG use nomic-embed-text.

This split is the subject of DL-01 (cost and stack) and DL-06 (why two Ollama models instead of one). It keeps text generation at zero API cost.

HOW PROGRESS REACHES THE PAGE

/generate streams Server-Sent Events as it runs — one event per step (vision, audience, copywriter, guardrails, cta) and a final "done" event with the run id. The page shows the pipeline advancing live instead of freezing on a spinner, which matters because a run takes roughly a minute. The finished output is kept in memory under that run id and one metrics line is appended to run_log.jsonl.

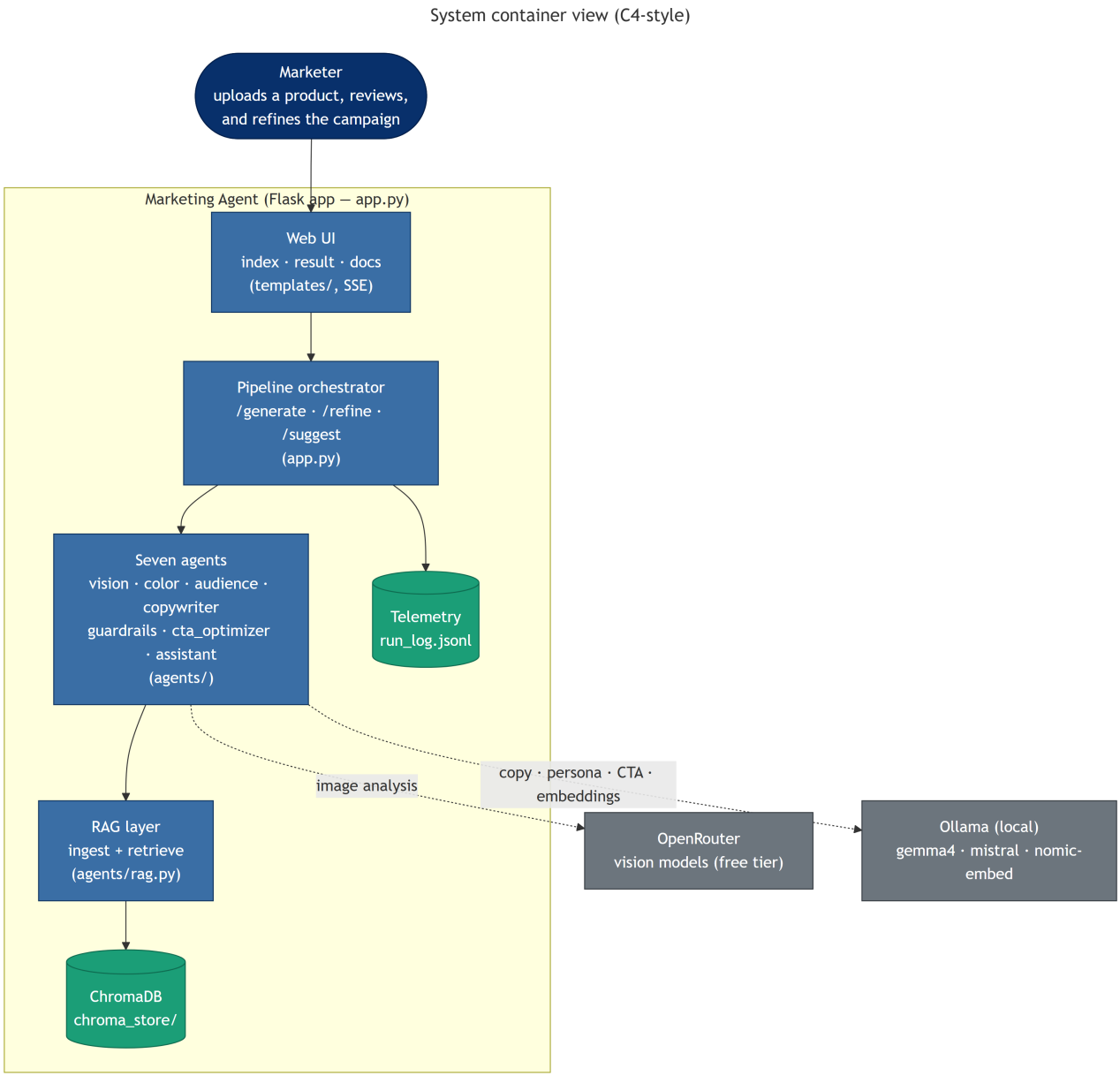


See the picture: diagrams/campaign-lifecycle.mmd

WHERE THE WHOLE THING SITS

The Flask app is the only moving part the user sees. Inside it are the web UI, the orchestrator, the seven agents, and the RAG layer; outside it are two services it calls —

OpenRouter for vision and a local Ollama for text and embeddings — plus ChromaDB and the telemetry log on disk.



See the picture: diagrams/system-c4-container.mmd

WHAT HAPPENS IF A STEP BREAKS

The run is wrapped so a failure becomes an "error" SSE event with a plain message rather than a crash. Vision has its own safety net: if the primary model returns an error, the fallback model is tried before the step gives up. Because the output is only stored after a clean run, the result page redirects home if asked for a run id it does not have (for example after a server restart, since runs are kept in memory).

WHY IT'S BUILT THIS WAY

The fixed order and narrow agent scopes are the mechanism the GAP analysis asks for: five agency functions, each with the right context at the right time, with a person kept for review. Running them as a visible, streamed sequence also satisfies the brief's

requirement that the collaboration be legible — you can see which agent is working and, on refinement, which agents re-ran.

WHERE THIS LIVES IN THE CODE

- app.py the /generate orchestrator and SSE stream
- agents/vision.py step 2, product image -> tags and signals
- agents/color.py step 1, image -> palette
- agents/rag.py step 3, retrieve document context
- agents/audience.py step 4, signals -> persona
- agents/copywriter.py step 5, persona -> two variants
- agents/guardrails.py step 6, regex claim checks
- agents/cta_optimizer.py step 7, CTA scoring
- config.py the model split (OLLAMA_MODEL / OLLAMA_FAST_MODEL)
- run_log.jsonl one metrics line per run

DIAGRAMS FOR THIS DOCUMENT

- diagrams/pipeline-overview.mmd the run, left to right
- diagrams/pipeline-dataflow.mmd each agent's inputs and outputs
- diagrams/campaign-lifecycle.mmd one run as a state machine
- diagrams/system-c4-container.mmd how all the parts fit together